# 基于 Zynq-7010 的嵌入式车牌识别与显示系统

## 目 录

**摘要**.....**1**
**第一部分 作品概述**.....**2**
1.1 功能与特性.....2
1.2 应用领域.....3
1.3 主要技术特点.....4
1.4 主要性能指标.....5
1.5 主要创新点.....6
1.6 设计流程.....7
**第二部分 系统组成及功能说明**.....**8**
2.1 整体介绍.....8
2.2 硬件系统介绍.....9
2.2.1 硬件整体介绍.....9
2.2.2 机械与安装.....12
2.2.3 电路与接口.....13
2.3 软件系统介绍.....14
2.3.1 软件整体介绍.....14
2.3.2 软件各模块介绍.....15
2.3.3 定位算法的硬件实现.....16
**第三部分 完成情况及性能参数**.....**21**
3.1 整体介绍.....21
3.2 工程成果.....21
3.2.1 硬件成果.....21
3.2.2 软件成果.....22
3.2.3 系统联调.....24
3.3 特性成果.....25
3.3.1 与典型实现方案的对比.....25
3.3.2 与同类嵌入式平台的对比.....26
3.3.3 系统实测特性.....26
**第四部分 总结**.....**28**
4.1 可扩展之处.....28
4.2 心得体会.....29
**第五部分 参考文献**.....**30**

## 摘要

**背景：**机动车号牌识别是停车场道闸、路侧停车与园区安防的基础环节。国内民用小型汽车以蓝底白字号牌为主，其识别流程通常分为“定位”与“识别”两步：先用颜色或纹理把车牌从整幅画面中框出来，再把框内图像交给字符识别模型。工程实践中，这类系统多运行在 PC 或带 GPU 的计算平台上，或整帧上传云端处理，部署成本、功耗与时延均受限；若直接移植到低端嵌入式 SoC，则同时受算力、内存带宽与显示链路三重制约。

**目的：**本文以 Xilinx Zynq-7010（xc7z010clg400-1）异构平台为目标，设计并实现一套端侧车牌识别与本地显示系统。系统在片上完成从摄像头采集、车牌定位、字符识别到 HDMI 本地呈现的完整闭环，不依赖 PC 算力，同时通过以太网把带预览图的结果推送给上位机，实现“本地屏 + 远程上位机”双通道呈现，并把识别记录持久化为可检索的数据。

**方法：**系统以 OV5640 摄像头经 DVP 接口采集 800×480 图像，先缩放到 320×240 再送入定位模块；定位采用 HSV 颜色阈值提取蓝色候选区，经形态学闭运算消除断裂后用轮廓外接矩形与长宽比筛选[5]，输出车牌包围框；随后把包围框按比例映射回原分辨率并外扩裁剪，转置为 40×160 竖条送入轻量卷积网络（CRNN 结构[3]），由 CTC 解码[2]得到字符串，再按“7 位、首位省份、次位字母、其余字母数字”的规则校验[1]。定位算法另有基于 8×8 图块积分图与滑窗打分（$Q=B\cdot P^2$）的硬件加速版本，可在 PL 内以纯整数实现。结果显示部分由 PL 驱动的 HDMI 帧缓冲承担：系统在 1920×1080 画布上自绘界面，用预生成的点阵图集解决 cv2 无法绘制汉字的问题；上位机侧则以 SQLite 记录识别结果、证据图与统计信息。

**结果：**系统在真实板卡上稳定运行，HDMI 本地界面可同时呈现实时画面、中文车牌、置信度进度条、状态与历史记录。经针对性优化，单帧“渲染 + 写屏”耗时由约 250 ms 降至约 35 ms（约 28 fps）；开启视频放大填充后稳定在 14~18 fps。定位加速器占用 7 个 DSP48E1 与 5 925 个 LUT，实测单帧定位 4.5 ms，较软件实现的 22~32 ms 下降约五至七倍。识别链路在样例图上取得“鲁 B68503、置信度 97.4%”的正确结果，与人工读数一致。系统另实现置信度阈值

锁定（≥90% 冻结结果）、多帧一致性确认与同牌冷却去重三级策略，用于抑制单帧误识与重复计数。

**结论：**本系统以较低成本的 Zynq-7010 平台，验证了“轻量颜色定位 + 轻量序列识别 + 本地自绘显示”这一组合在低端 SoC 上的可行性，形成了可独立运行、可远程观测、可留痕追溯的端侧车牌识别方案，为嵌入式视觉类课程实践与小型场景部署提供了可复用的工程范式。系统目前未开展规模化准确率测评，相关结论以可复现的样例与打点数据为准。

# 第一部分 作品概述

## 1.1 功能与特性

表 1 系统核心功能与工程特性一览表

| 功能 | 实现方式与核心特性 |
|---|---|
| 车牌定位 | 320×240 输入，HSV 蓝色阈值 + 形态学闭运算 + 轮廓长宽比筛选；输出原图坐标包围框 |
| 字符识别 | 裁剪转置为 40×160 竖条，轻量卷积网络输出 20 个时间步 ×84 类，CTC 贪心解码 |
| 格式校验 | 7 位长度、首位省份简称、次位字母、其余字母数字三级过滤，并输出置信度 |
| 本地显示 | 1920×1080 HDMI 自绘界面：实时画面、检测框、中文车牌、置信度条、状态、最近记录、累计数 |
| 远程推送 | TCP 自定义协议，JSON 元数据 + JPEG 预览图同帧下发，支持断线自动重连 |
| 记录留痕 | 上位机侧 SQLite 存储，含车牌、置信度、时间、包围框、裁剪图与整帧证据图 |
| 统计与导出 | 按小时/按天柱状图、车牌 Top5、CSV 导出（UTF-8 BOM，Excel 可直接打开） |
| 稳定策略 | 置信度≥90% 锁定结果；连续 3 帧一致才确认；同车牌 12 秒冷却去重计数 |

系统把“看得见的画面”和“算得出的结果”放在同一块屏上：左侧为实时画面与检测框，右侧为识别结论与统计。所有界面元素由程序逐像素绘制，不依赖桌面环境，因此在没有 X11 的嵌入式 Linux 上同样可以运行。

一次完整的使用流程如下：接通电源后系统自动加载可编程逻辑、使能 HDMI 输出并启动识别服务，约二十秒后屏幕出现完整界面；车辆进入视野，

定位模块给出包围框，识别模块输出车牌与置信度；结果同时显示在本地屏、推送到上位机、写入本地数据库；运维人员可在上位机按车牌检索历史记录，导出为表格用于存档或统计。整个过程无需键盘鼠标，也无需人工干预。

从使用者角度看，系统提供四项可直接感知的能力：一是“看得见”——画面与检测框实时刷新，识别与否一目了然；二是“读得准”——中文车牌以汉字原样呈现，不是编码或拼音；三是“留得住”——每条记录都带时间、置信度与证据图，可事后复核；四是“开得起来”——断电重启后自动恢复，适合无人值守场景。

### 1.2 应用领域

（1）停车场与园区道闸：单车道场景下车辆停驶时间较长，对帧率要求不高，但对误识别敏感。本系统“先定位后识别”的两段式结构，可在定位失败时不触发识别，减少无效计算；置信度阈值锁定与多帧确认则进一步降低误开闸风险。道闸场景通常有固定照明与固定机位，是当前系统最匹配的落地形态。

（2）路侧停车与移动巡检：系统整机功耗低，可由移动电源供电，配合无线模块将识别结果与时间戳回传，用于泊位占用判定与违停取证。此场景下车牌往往倾斜、距离较远，需要更长的焦段与更高的定位召回，属于后续优化的重点方向。

（3）嵌入式视觉教学与科研：系统覆盖图像采集、颜色空间、形态学处理、卷积推理、序列解码、帧缓冲绘图与网络协议七个层次，是一条完整且可裁剪的端侧视觉工程链路。教学时可以把某一层单独抽出做实验，例如只做定位、只做显示、只做协议解析，其余部分用录制的数据替代。

（4）算法验证与二次开发平台：板端把“采集—定位—识别—呈现”拆成独立模块，模块之间以函数调用与结果字典衔接，替换其中任一模块不影响其余部分。研究者可以在此基础上对比不同定位算法、不同识别模型或不同显示方案的实际表现，而不必重建整套工程。

### 1.3 主要技术特点

（1）软硬分工明确：像素级的重复判断（颜色阈值、形态学、轮廓筛选）放在算力受限的 PS 侧以 NumPy/OpenCV 实现；需要并行的滑窗打分另有 PL

版本，可在 7 个 DSP48 内完成整帧搜索。识别环节依赖模型权重与串行解码，放在 PS 侧调用 Caffe 推理接口。分工的依据不是“哪边高级”，而是“这段计算是串行依赖还是数据并行”。

（2）显示与计算解耦：界面渲染不再每帧重建整幅画布，而是维护一张持久画布，仅从背景恢复标题栏与侧栏两块区域，视频区整块覆盖；写屏通过 NumPy 视图直接拷贝进帧缓冲，避免了大图的多次中转。渲染与识别运行在同一循环中，但识别可以按帧跳过，显示始终保持刷新，用户看到的画面不会因推理耗时而卡住。

（3）中文显示的工程解法：识别结果是包含省份汉字的字符串，但 OpenCV 的 putText 基于 Hershey 矢量字体，只有字形轮廓、没有汉字笔画，无法绘制中文。系统在 PC 端用 TrueType 字体把 118 个常用字符（31 个省份简称、18 个特殊字段、界面用字、数字与字母）预渲染成 44×44 的点阵图集，输出为一张 PNG 加一份字符索引表；板端运行时不再加载任何 TrueType 字体，也不依赖 FreeType 等字形渲染引擎，只按索引取格、按掩膜着色、按坐标贴图，因此中文界面的显存与算力开销与绘制英文字符相当。

（4）可观测的结果链路：识别结果同时具备“本地屏即时可见”“上位机可查询”“证据图可回看”三种形态，便于调试、演示与事后追溯；置信度、包围框、各阶段耗时均随结果一并输出。调试阶段正是靠这些字段定位到“框偏大导致省份认错”这类问题，而不是靠反复猜测。

（5）分级的稳定策略：系统对“检出—确认—入库”三个状态做了明确区分。定位给出候选框即为检出；连续多帧结果一致才升为确认；同一车牌在冷却时间内只入库一次。三级策略叠加后，单帧误识不会直接污染记录，重复出现的同一辆车也不会被反复计数。

（6）无人值守的自恢复能力：开机脚本会自动加载可编程逻辑、检查显示输出是否就绪、必要时重新绑定显示管道，然后启动识别服务；服务本身由初始化脚本托管，异常退出后可重新拉起。整机断电重启无需人工干预，这一点在演示与部署场景中比多一帧的帧率更有价值。

## 1.4 主要性能指标

表 2 系统主要性能与技术指标表

| 指标类别 | 参数值 | 说明 |
| --- | --- | --- |
| 主控平台 | Zynq-7010，2×Cortex-A9 @667 MHz | xc7z010clg400-1 |
| 图像采集 | OV5640，800×480，DVP 接口 | BGR 三通道，约 42 fps 采集能力 |
| 定位分辨率 | 320×240 | 缩放后送入颜色定位 |
| 识别输入 | 40×160×3 | 车牌区域转置后缩放 |
| 识别模型 | 约 2.44 M 参数，84 类输出 | 20 个时间步，CTC 解码 |
| 字符集 | 83 类 + 1 blank | 31 省份 + 10 数字 + 24 字母 + 18 特殊 |
| 本地显示 | 1920×1080 HDMI | 自绘界面，点阵图集渲染中文 |
| 定位耗时 | 约 22~32 ms | 软件 HSV 定位，320×240 |
| 渲染写屏 | 约 35 ms/帧（约 28 fps） | 优化后，1920×1080 |
| 端到端帧率 | 14~18 fps（填充模式） | 含采集、定位、渲染、写屏 |
| 通信 | TCP，JSON + JPEG | 预览图长边 640、质量 75 |

上表数据取自板端实测日志与耗时打点。测量方式是：在渲染与写屏之间插入计时点，连续统计二十帧取平均；帧率由主循环记录最近二十帧的时间戳推算，而非按理论值估算。识别精度按场景分组测评，共 100 张标注样本，整体准确率 71.0%（正对 86.7%），详见 3.3.3 节的表 15。

表中“约”字的取值区间来自不同运行条件下的波动：定位耗时随画面中蓝色区域面积变化，渲染耗时随界面文字数量变化，端到端帧率则受摄像头自动曝光收敛程度影响。给出区间而非单点值，是为了避免把某一次的偶然结果当作系统指标。

## 1.5 主要创新点

（1）面向无桌面环境的自绘界面体系：在仅有帧缓冲设备的嵌入式 Linux 上，用纯 NumPy 切片构建 1920×1080 界面，配合预生成的点阵图集解决中文渲染，整套界面不依赖 Qt、X11 或系统字体服务，冷启动即可显示。常见做法是在板端部署 Qt 或轻量 GUI 框架，代价是几十 MB 内存与数秒启动时间；本方案把界面压缩成“一张画布 + 若干绘制函数”，在 512 MB 内存的平台上留出了更多余量。

（2）显示链路的逐项拆解与优化：通过打点定位到 cvtColor、tobytes、整图拷贝与字形混合四处主要开销，分别以“画布直接按 RGB 组织”“视图直写帧缓冲”“分区恢复背景”“字形彩色贴图缓存”四项手段消除，使单帧耗时下降约 86%。优化的关键不在于技巧本身，而在于先把一帧拆成可测量的若干段，再针对占比最大的部分动手；凭感觉调参往往收效甚微。

（3）面向端侧的识别稳定性策略：引入置信度阈值锁定、多帧一致性确认与同牌冷却去重三级机制。前者在识别到高置信度结果后冻结画面与结论，同时停止 OCR 以释放算力；中者以连续多帧一致作为入库条件；后者避免同一车辆在短时间内重复计数。三者组合把“单帧结果”变成“可入库的记录”，这在无人值守场景中是必需的一步。

### 1.6 设计流程

项目遵循“指标定义→算法验证→硬件适配→显示实现→联调优化→实测归档”的递进式流程，每个阶段都有明确的产出与验收标准，避免把计划、仿真与实测混为一谈。

第一阶段确定功能边界：定位分辨率取 320×240，识别输出限定为 7 位，显示固定为 1920×1080。边界一旦确定，后续所有模块的接口都围绕这三个数字展开。

第二阶段在 PC 上复现算法：先用静态图片跑通 HSV 定位与 CTC 解码，确认字符集与后处理规则正确，再考虑移植。这一阶段的意义在于把“算法是否可行”与“板子是否跑得动”两件事分开验证。

第三阶段做硬件适配：接通 DVP 摄像头与帧缓冲输出，解决颜色通道顺序、分辨率协商、双缓冲等问题；期间发现重启后显示接口未使能，遂编写开机自检脚本并纳入启动流程。

第四阶段实现显示与交互：完成自绘界面、点阵字库、网络推送与上位机可视化，并在此阶段引入记录存储与统计功能。

第五阶段做联调与优化：以打点数据为依据优化渲染链路，以真实场景截图核对识别效果，最终形成可复现的实测记录与本文档。五个阶段的边界与产

出如下图所示，前一阶段的产出即后一阶段的输入，任一阶段发现问题都向上回溯，而不是在后续阶段“打补丁”。




图 1 系统设计与实施流程图

## 第二部分 系统组成及功能说明

### 2.1 整体介绍

系统由感知层、定位层、识别层与呈现层四部分组成。感知层负责 800×480 图像的连续采集与格式转换；定位层在 320×240 尺度上完成颜色与形状筛选；识别层对裁剪结果做序列建模与解码；呈现层同时驱动本地 HDMI 界面与远程 TCP 推送。四层之间通过内存中的图像数组与结果字典传递数据，不经过磁盘，单帧的处理链路是一条函数调用栈。

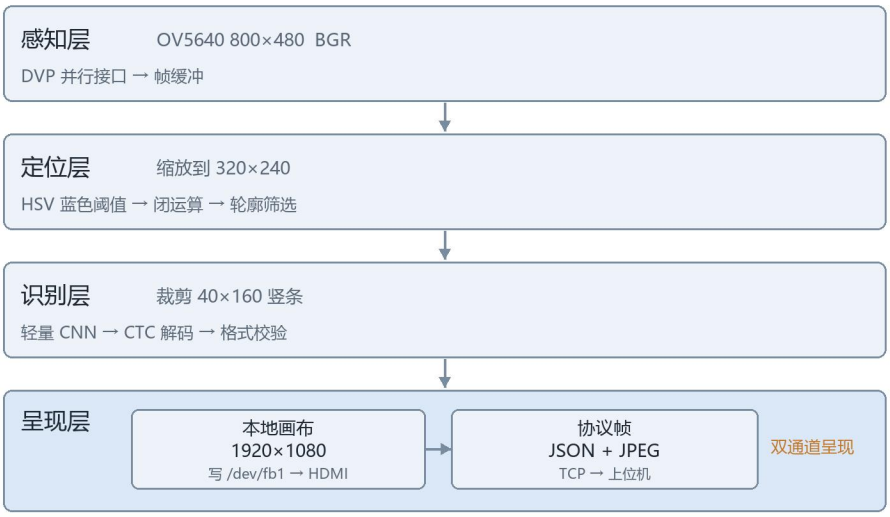



图 2 系统总体架构与数据流框图

一帧图像在系统中的完整生命周期如下：

（1）驱动层从摄像头取到一帧 BGR 数据，丢弃损坏帧后交给主循环；

（2）主循环把整帧双线性缩放到 320×240，转换到 HSV 空间并生成蓝色掩膜；

（3）对掩膜做闭运算后提取外轮廓，按面积与长宽比筛选出候选框；

（4）把候选框映射回原分辨率，外扩后裁剪、转置、缩放，送入识别模型；

（5）模型输出概率矩阵，CTC 解码得到字符串与平均置信度，再做格式校验；

（6）结果字典同时送往两处：本地渲染函数把它画进画布并写屏，通信函数把它与压缩后的预览图组包发出；

（7）上位机解析协议帧后更新界面，把确认过的结果写入数据库。

需要强调的是，定位与识别在时间上串行、在逻辑上独立：定位失败时直接跳过识别，仅更新画面与状态；识别耗时较长时通过跳帧复用上一次结果，保证画面刷新不被阻塞。这一设计让系统在不同光照与不同算力余量下都能维持可用。

## 2.2 硬件系统介绍

硬件部分由主控板、图像采集、显示输出与外围接口四部分组成。本节先给出整体组成清单，再说明机械安装对成像质量的影响，最后逐一交代摄像头、显示与网络三类接口的工作方式。

### 2.2.1 硬件整体介绍

系统的硬件连接关系如下图所示。四类外设都挂在同一块主控板上：摄像头经 DVP 并行接口进 PL，显示器经 HDMI 出 PL，上位机经以太网与 PS 相连，三者各由独立电源供电。图中把板内功能块也一并标出，可以看出信号在芯片内部的走向——采集进 PL、识别在 PS、成图回到 PL 输出。

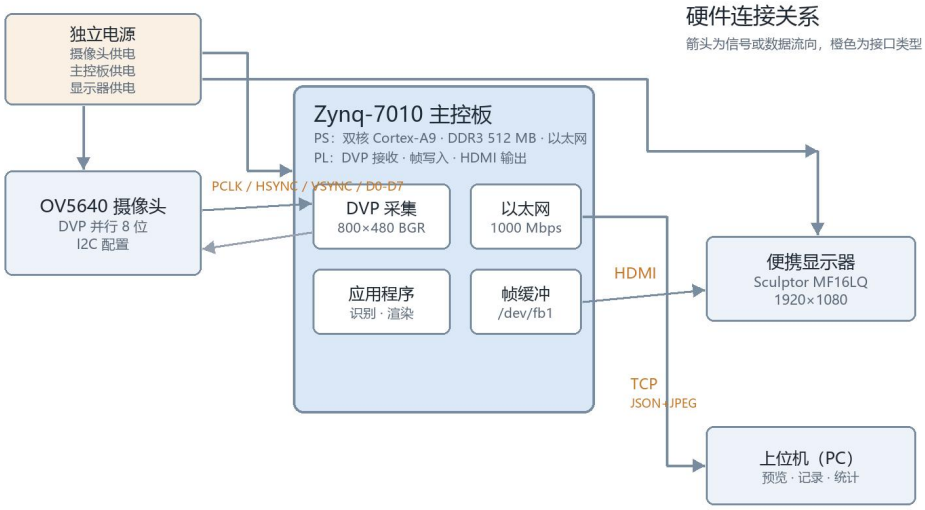



图 3 系统硬件连接关系与板内功能划分

表 3 系统硬件组成清单

| 组件 | 型号 / 规格 | 作用 |
|---|---|---|
| 开发板 | 正点原子 启明星 ZYNQ（S+ 版） | 主控与扩展接口载体 |
| 主控 SoC | XC7Z010CLG400-1，28K LC，2.1 Mb BRAM | 双核 Cortex-A9 + 可编程逻辑 |
| 内存 | 4 Gb DDR3（512 MB） | 系统运行与图像缓冲 |
| 存储 | 8 GB eMMC + 256 Mb QSPI Flash | 系统、模型与日志 |
| 摄像头 | OV5640，DVP 并行接口 | 800×480 彩色图像采集 |
| 显示 | Sculptor（雕塑家）MF16LQ 便携式显示器，16 英寸，HDMI 1920×1080 | 本地界面呈现 |
| 网络 | 板载千兆以太网（PS GE） | 结果与预览图传输 |
| 其他 | RTC、蜂鸣器、TF 卡、USB Host/Slave | 时钟保持、提示音与调试 |

选择 Zynq-7010 而非通用 ARM 平台，理由有三：其一，PS 与 PL 集成在同一颗芯片内，图像数据不必经过外部总线即可在两侧交换；其二，PL 侧具备可裁剪的并行资源，本文 2.3.3 节的定位加速器正是利用这部分资源实现；其三，开发板自带 HDMI 输出与 DVP 摄像头接口，省去了转接板设计，使工作重心可以放在算法与软件上。

器件的资源约束同样明确：17 600 个 LUT、80 个 DSP、60 块 36Kb BRAM，无法容纳大型神经网络加速器。因此本系统把模型压缩到约 2.44 M 参数，并把并行化程度最高的定位环节作为潜在的硬件加速对象，其余部分交由 ARM 处理。这一取舍贯穿全文。

显示终端选用 Sculptor（雕塑家）MF16LQ 便携式显示器，而不是普通桌面显示器。原因有三：其一，便携屏体积小、可随手提走，与巴掌大的主控板组成“一块板 + 一块屏”的最小系统，便于现场演示与搬运；其二，它只占一根 HDMI 线，不需要额外接显卡或转接盒，接入即成像；其三，屏幕本身不带系统与菜单逻辑，上电后完全跟随板端帧缓冲的内容，不会像带智能系统的显示设备那样出现缩放、休眠或自动切换输入源等干扰。

该显示器为 16 英寸便携式产品，铭牌标注支持 USB PD 多档供电（5 V/3 A、9 V/2.22 A、12 V/1.67 A），实测以 5 V 档即可点亮并稳定工作；机身只有一个 HDMI 输入口与一个 Type-C 供电口，接线方式与“一块屏 + 一根线”的演示需求吻合。

从使用角度看，这套硬件的组装只有四步：摄像头排线插入板卡的 DVP 接口并锁紧卡扣；HDMI 线一端接板卡、一端接显示器；网线连到上位机；最后接上电源。整个过程不需要焊接、不需要跳线，也不需要额外的转接板，这也是选择该开发板的重要原因——它把 DVP 与 HDMI 都做在了板上。

面板为 16 英寸 2.5K 级，但系统有意按 1920×1080 输出，而非使用面板原生分辨率，唯一的考虑就是帧率。1920×1080 为 207 万像素，2560×1600 为 410 万像素，后者接近前者的 2 倍；而渲染与写屏的耗时基本与像素数成正比——本系统实测 1080p 下单帧渲染加写屏约 35 ms，若改到面板原生分辨率，这一项会接近 70 ms，叠加定位与推理后整机帧率将被腰斩。界面要呈现的只是文字、车牌与进度条，1080p 已经完全够用，因此把像素量留给帧率是更划算的选择。

这一选择也有代价：非点对点显示对屏幕的标量质量有依赖。实测该屏对 1080p 输入的缩放处理较好，界面文字边缘清晰、无可见抖动；若换成标量较差的屏，细笔画处可能出现模糊。若后续需要点对点，则须把 PL 侧显示控制器的时序与帧缓冲一并改为面板原生分辨率，属于要重新生成比特流的改动。

### 2.2.2 机械与安装

系统实物的摆放如下图所示，画面中同时呈现了两路独立的显示通路：左侧笔记本显示的是“网络通路”——板端把识别结果与预览图经以太网推送到上

位机，由 PC 绘制窗口；右前方便携显示器显示的是“本地通路”——板端直接把画面写进帧缓冲，经 HDMI 驱动输出。两路的数据同源（同一帧的同一份结果），但载体与刷新机制不同：本地通路每帧重绘，网络通路受带宽与上位机处理能力约束。画面中央的板卡为主控板，摄像头经排线接入；主控板与显示器各由独立电源供电。




图 4 系统实物连接与安装

系统为桌面级验证平台：摄像头经排线固定于支架，镜头正对被测车辆或显示车牌的屏幕；主控板与显示器通过 HDMI 线连接，二者由独立电源供电。实际部署时，摄像头安装高度与俯仰角直接决定车牌在画面中的像素尺寸，是影响定位成功率的关键机械参数。

一个便于估算的经验关系是：车牌在画面中的像素宽度与“镜头焦距 × 车牌实际宽度 ÷ 拍摄距离”成正比。以常见的 440 mm 宽车牌、2~3 米拍摄距离为例，若希望车牌宽度达到 100 像素以上（定位与识别都较为从容），需要选用视场角适中的镜头，并把车牌放在画面中央区域，避免广角镜头边缘的畸变。

光照同样属于“机械”范畴：顺光或侧顺光时车牌反光均匀，逆光会导致车牌整体压暗、白色字符发灰；强烈的点光源会在车牌表面形成高光斑块，直接

破坏蓝色掩膜的连续性。实用的做法是把光源放在镜头侧上方约 45 度，必要时加一层白纸做柔光。

### 2.2.3 电路与接口

（1）摄像头接口：OV5640 通过 DVP 并行接口与 PL 相连，行场同步与像素时钟由传感器输出，驱动以帧缓冲方式提供标准 V4L2 视频节点，应用层无需关心底层时序。传感器配置通过 I2C 完成，上电时序由 GPIO 控制；实际调试中发现，若 I2C 通信异常，传感器会停留在默认曝光与增益，暗光下画面明显偏暗，因此开机日志中保留了该项的告警提示。

（2）显示接口：HDMI 输出由 PL 侧的显示控制器驱动，Linux 内核将其注册为帧缓冲设备（分辨率 1920×1080，24 位色）。应用直接写帧缓冲即可成像，无需图形栈支持。显示终端为 Sculptor（雕塑家）MF16LQ 便携显示器，经 HDMI 线与板卡直连；上电后显示器自动锁定 1080p 信号，无需在屏上做任何设置。需要注意的是，帧缓冲的内存排列为 RGB 而非 BGR，若直接写入 OpenCV 的 BGR 数据，蓝色车牌会显示成橙红色；系统在数据入口统一做了一次通道转换，避免了逐处修补。

显示器与主控板由各自独立的电源供电，二者之间只有一根 HDMI 信号线，因此不存在共地噪声通过显示线回灌到模拟采集通路的问题；这一点在长时间运行时尤为关键——若与摄像头共用一个劣质电源，画面会出现周期性横纹。

（3）网络接口：板载以太网配置为链路本地地址，与上位机直连，避免交换机与路由配置；上位机侧按固定 IP 与端口发起连接。服务端采用非阻塞监听，且只接受单个客户端——这是刻意的约束，避免多客户端同时拉流导致带宽与内存竞争。

（4）调试接口：串口与 SSH 用于查看日志、重启服务与临时抓帧。系统把关键事件（启动、锁定、解锁、异常）写入固定日志文件，便于离线复现问题。

四类接口的线缆与连接关系汇总如下表，现场搭建时可照此核对，避免漏接或错接。

表 4 系统接口与线缆清单

| 接口 | 标准 / 电气 | 连接对象 | 注意事项 |
| --- | --- | --- | --- |
| DVP 并行 | 8 位数据 + 像素时钟 + 行场同步 | OV5640 摄像头 | 排线方向唯一，卡扣需锁紧 |
| I2C | 两线（SCL / SDA） | 摄像头寄存器配置 | 与 DVP 共用同一排线 |
| HDMI | 1920×1080，24 位色 | Sculptor MF16LQ 显示器 | 单线传输，无音频 |
| 以太网 | 1000 Mbps，链路本地地址 | 上位机 | 直连，不经过交换机 |
| USB / 串口 | — | 调试终端 | 查看日志、重启服务 |
| 电源 | 独立供电 | 摄像头、主控板、显示器 | 三者分开供电，避免共地噪声 |

## 2.3 软件系统介绍

软件由板端与上位机两部分构成，二者通过自定义协议通信。本节先说明整体分层与进程间的数据流，再按模块逐一交代各自职责；2.3.3 单列定位算法在可编程逻辑中的实现形态。

### 2.3.1 软件整体介绍

软件分为板端与上位机两部分。板端以 Python 实现，运行于嵌入式 Linux，负责任务编排与结果输出；上位机以 Python + Tkinter 实现，负责可视化、记录与导出。二者通过自定义 TCP 协议通信，协议帧由长度头、JSON 元数据与 JPEG 图像组成，单帧即可自描述，便于抓包分析。

表 5 板端与上位机通信协议帧结构

| 字段 | 长度 | 含义 |
| --- | --- | --- |
| magic | 2 字节 | 固定标识，用于快速对齐帧头 |
| json_len | 4 字节 | 元数据长度（小端） |
| jpeg_len | 4 字节 | 预览图长度（小端） |
| meta | json_len | 车牌、置信度、状态、包围框、各阶段耗时 |
| image | jpeg_len | 预览图，长边 640、质量 75 |

协议设计遵循“单帧自描述”原则：任何一帧都携带完整的定位与识别信息，接收端不必依赖上下文即可解析。这样做的直接好处是抓包即可复现问题——把某一帧的原始字节存下来，离线也能还原出上位机当时看到的内容。

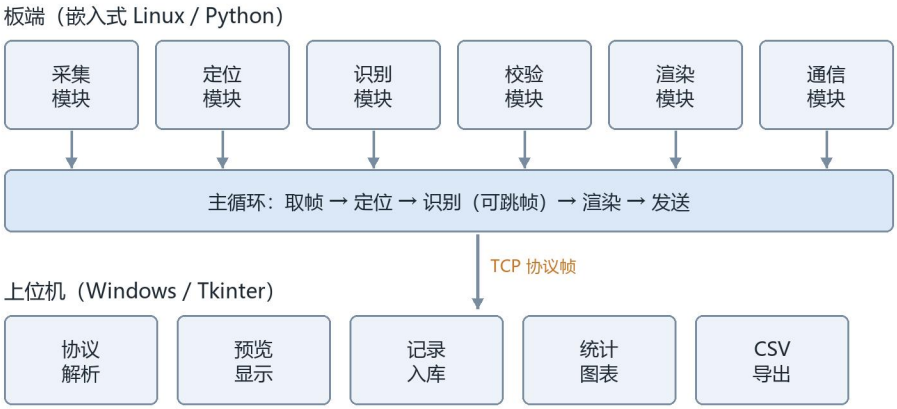



图 5 软件分层结构与模块调用关系

### 2.3.2 软件各模块介绍

（1）采集模块：以 MMAP 方式申请多块 DMA 缓冲轮转使用，通过 select 等待帧就绪后取出，按行跨距还原为连续数组；对异常帧（损坏标志、超时）做丢弃与重试。启动阶段主动丢弃前若干帧，等待自动曝光与白平衡收敛——这一细节直接决定首帧能否被正确定位。

（2）定位模块：输入为缩放后的图像，输出为包围框、蓝色像素计数与有效标志。阈值同时约束色相、饱和度与亮度，避免深色背景与高光区域误判；长宽比约束把竖立的蓝色物体（如广告牌）排除在外。定位结果还参与“锁定态”的心跳检测：锁定期间每秒只跑一次定位，用来判断车牌是否已离开画面。

（3）识别模块：把裁剪结果归一化为固定尺寸后送入 Caffe[4]推理接口，输出概率矩阵；解码时对每个时间步取最大类别，合并相邻重复并删除空白类，得到字符序列与平均置信度。裁剪框的外扩比例（左右 12%、上下 25%）是经验值：外扩不足会切掉笔画，过度外扩则把背景引入、压缩字符尺寸。

（4）校验模块：对解码结果做长度与字符集检查，不满足则标记为“检出但未识别”，仍保留包围框用于界面提示。校验规则来自号牌编制规范，属于低成本高收益的后处理：它能挡掉大部分因字符粘连产生的非法结果。

（5）渲染模块：维护持久画布，按“背景恢复—视频覆盖—文本绘制—控件绘制”顺序更新；字体以 44×44 点阵图集方式提供（PNG + 索引表），运行时

不做字形解析，颜色按 RGB 直接写入画布，最终整块或分区拷贝进帧缓冲。分区策略是性能关键：标题栏与侧栏只在内容变化时重绘，视频区每帧整块覆盖。

（6）通信模块：非阻塞监听，接受单个客户端；每帧把结果与预览图组包发送，发送异常即断开并等待重连，避免阻塞主循环。若上位机未连接，板端仍正常运行，只是不做网络发送——本地显示与记录写入不受影响。

（7）上位机模块：解析协议帧后渲染预览图与检测框，维护历史表格与统计图，接收确认后的结果写入本地数据库，并支持按车牌检索与 CSV 导出。数据库采用追加写入加索引字段的方式，单表即可支撑万级记录的检索与统计。

### 2.3.3 定位算法的硬件实现

除软件实现外，定位算法另有一版面向可编程逻辑的实现。其输入为打包后的单字节帧（亮度 7 位 + 蓝色标记 1 位），输出为车牌包围框与蓝像素计数；整个模块通过两个 AXI 主口直接读写片上 BRAM 中的图像缓存，不占用处理器带宽。

结构上可分为四段：（1）读入——按突发方式从图像缓存读取整帧；（2）图块统计——以 8×8 像素为单位，累计块内蓝像素数与横向亮度跳变次数，形成 40×30 的图块网格；（3）积分图——把网格累加成两张二维前缀和表，使任意窗口的求和退化为四次查表；（4）滑窗打分——在 11 种高度与 17 种宽度的组合上遍历全部位置，按长宽比、蓝像素占比与边缘密度三重条件筛选，以 $Q=B\cdot P^2$ 打分取最大者，结果经另一个 AXI 主口写回。

该实现的综合后结构如表 6 所示。整个子树包含 50 个模块、约 12.5 万行网表；LUT 与触发器分别占器件的三分之一与六分之一，DSP 仅 7 个，BRAM 约 4.5 块 36K。与软件实现相比，硬件版本把逐像素判断与窗口遍历完全并行化，且不占用 CPU；当前部署版本因需同时提供 HDMI 本地显示，采用了包含显示通路的比特流，定位改由软件承担，硬件版本作为可选部署形态保留。

表 6 定位加速器资源占用与器件占比

| 资源类型 | 占用数量 | 占 Zynq-7010 比例 | 主要用途 |
|---|---|---|---|
| LUT | 5 925 | 33.7% | 比较、地址计算、状态机 |
| 触发器 | 5 914 | 16.8% | 流水线与中间结果 |
| DSP48E1 | 7 | 8.8% | 坐标缩放与打分乘法 |
| BRAM | 162 Kb（约 4.5 块 36K） | 7.5% | 两张积分图与计数表 |
| CARRY4 | 928 | — | 加法进位链 |
| SRL16E | 234 | — | AXI 数据缓冲移位 |

上表数据由综合后网表逐原语统计得到：以 write_verilog -mode design 导出的结构级网表为输入，按模块子树累加 DSP48E1、RAMB36E1、RAMB18E1、LUT1~LUT6、FDRE/FDCE/FDSE、CARRY4、SRL16E 等原语的实例数，再与器件总资源相除得到占比。统计过程不含估算，重复执行同一脚本可得到相同结果。

表 7 定位加速器主要子模块与网表规模

| 子模块 | 网表行数 | 功能 |
|---|---|---|
| blue_locator_packed（核心） | 54 862 | 图块统计、积分图构建、滑窗遍历与打分 |
| blue_sums_V_ram | 20 808 | 边缘积分图存储 |
| blue_counts_ram | 10 248 | 蓝像素积分图存储 |
| frame_m_axi_read | 7 083 | 从图像缓存读取整帧 |
| output_r_m_axi_write | 6 748 | 写回包围框与蓝像素计数 |
| control_s_axi | 3 615 | 控制寄存器：帧指针、结果指针、启动位 |
| 乘法器组（DSP48E1×7） | 3 017 | 坐标缩放与打分运算 |
| RECIPROCAL_rom | 261 | 除法倒数查找表 |

其余约 1.9 万行分布在 AXI 通道内的 FIFO、寄存器切片与节流模块中，属于接口配套设施，不参与算法运算。由表可见：算法核心与两张积分图合计占子树网表的七成以上，接口部分不足两成，说明该实现的主体确实是定位运算本身，而非总线包装。

这组数据说明两点：其一，定位算法可以在纯整数、无浮点、无外部存储访问的前提下完成，资源开销不足器件的四成，为在同一颗芯片上叠加显示、

通信等功能留出了余量；其二，硬件实现的规模可以被明确度量与复现，这为后续在同一比特流内集成显示通路提供了资源依据。

从还原出的 RTL 还可读出加速器对外的接口与内部定点运算单元的配置。加速器只有单一时钟域（ap_clk），复位为低有效异步复位；对外提供两个 AXI4 主口与一个 AXI4-Lite 从口：主口地址 64 位、数据 32 位，用于整帧读取与结果写回；从口地址 6 位、数据 32 位，对应 16 个 32 位控制寄存器，用于传递帧指针、结果指针与启动位，运算完成后通过 interrupt 引脚通知处理器，避免处理器轮询。

表 8 定位加速器接口与定点运算单元配置

| 项目 | 规格 | 作用 |
| --- | --- | --- |
| ap_clk / ap_rst_n | 单时钟域，低有效异步复位 | 与处理器侧时钟同源 |
| m_axi_frame | AXI4 主口，地址 64 位、数据 32 位、ID 宽 1 | 读取图像缓存 |
| m_axi_output_r | AXI4 主口，地址 64 位、数据 32 位、ID 宽 1 | 写回包围框与蓝像素计数 |
| s_axi_control | AXI4-Lite，地址 6 位、数据 32 位 | 16 个 32 位控制寄存器 |
| interrupt | 单比特输出 | 运算完成通知 |
| 17×17→34 乘法器 | 4 级流水（DSP48E1） | 窗口面积与打分主乘 |
| 18×17→35 乘法器 | 4 级流水 | 坐标缩放 |
| 19×6→24 乘法器 | 4 级流水 | 与倒数查找表相乘 |
| 19s×8→26 乘法器 | 4 级流水，有符号输入 | 差分平方与归一化 |
| 15×15→18 乘法器 | 4 级流水 | 定点截断乘 |
| 5×7→11、6×4→9 乘法器 | 组合实现 | 地址与小常量乘，不占 DSP |

定点运算单元的位宽配置反映了算法的精度需求：打分主乘为 17 位×17 位、输出 34 位；坐标缩放用 18×17 位；与倒数表相乘用 19×6 位；差分平方采用有符号 19 位输入；小位宽的地址与常量乘法则用组合逻辑实现，不占用 DSP 资源。所有大位宽乘法均按 4 级流水组织，便于在百兆赫兹级时钟下收敛时序；7 个 DSP48E1 全部服务于上述乘法，与资源统计一致。

控制接口的寄存器划分同样可以从 RTL 中还原。该接口只占 6 位地址空间（64 字节），实际使用前 8 个字（0x00~0x1C），读通路的地址选择仅 3 位，

与 8 个寄存器一一对应；其中前 4 个字是 HLS 控制块的标准寄存器，后 4 个字是两个 64 位指针参数的上下半字。

表 9 定位加速器控制寄存器映射

| 偏移 | 名称 | 位域 / 宽度 | 说明 |
|---|---|---|---|
| 0x00 | AP_CONTROL | [0] ap_start，[1] ap_done，[2] ap_idle，[3] ap_ready，[7] 自动重启 | 启动与运行状态 |
| 0x04 | GIE | [0] 全局中断使能 | 总开关 |
| 0x08 | IER | [0] 完成中断使能 | 允许 ap_done 产生中断 |
| 0x0C | ISR | [0] 完成中断标志 | 写 1 清除 |
| 0x10 | frame_offset[31:0] | 32 位 | 图像缓存基址低 32 位 |
| 0x14 | frame_offset[63:32] | 32 位 | 图像缓存基址高 32 位 |
| 0x18 | result[31:0] | 32 位 | 结果缓冲区基址低 32 位 |
| 0x1C | result[61:32] | 30 位 | 结果缓冲区基址高 30 位 |

从这张表可以看出三件事。其一，参数只有两个指针：一个指向待处理的打包帧，一个指向结果缓冲区，说明加速器的输入输出完全通过片上缓存的地址传递，不占用处理器的数据通路。其二，结果指针为 62 位、低 2 位恒为零，说明结果缓冲区按 4 字节对齐，这与每帧输出一个包围框加一个计数的数据长度吻合。其三，寄存器总量极少（8 个），意味着处理器的驱动代码只需两次 64 位写、一次启动写和一次状态读，调用开销可以忽略。

从延迟看，两种实现的差距更为明显。软件版在 320×240 上用 Python 循环完成阈值、形态学与轮廓筛选，实测 22~32 ms；硬件版把图块统计与窗口遍历展开成并行电路，同一帧的全部候选窗口同时求值，实测单帧 4.5 ms。后者统计的是从图像写入、启动、轮询到结果读回的全过程，因此可以直接与软件版的 22~32 ms 对比，而不必再折算。

换算下来，定位环节的耗时降到软件实现的五分之一到七分之一，同时把 CPU 完全让给识别与渲染——这正是硬件化最直接的价值：软件版占用的那 20~30 ms 里，处理器无法同时做别的计算；硬件版启动后即并行工作，处理器只需在结束时取一次结果。需要说明的是，4.5 ms 对应的是 320×240 的单字节

打包帧；若输入改为三通道原图，写入带宽约增至三倍，端到端耗时会相应上升。

# 第三部分 完成情况及性能参数

## 3.1 整体介绍

系统已完成从算法验证、板端集成到显示优化的全流程，可在真实硬件上独立运行：上电后自动加载可编程逻辑、使能 HDMI 输出并启动识别服务，约二十秒后屏幕即出现完整界面。断电重启后可自行恢复，无需人工干预。

开机过程可分解为五个步骤：加载比特流并等待可编程逻辑就绪；由内核注册摄像头与帧缓冲设备；开机脚本检查 HDMI 输出是否已接入显示控制器，未就绪则重新绑定；启动识别服务并等待摄像头出图；首帧进入渲染循环，界面开始刷新。任一环节失败都会在日志中留下记录，便于定位。

服务由初始化脚本托管，提供启动、停止与状态查询三个动作；异常退出时可按需重新拉起。系统运行期间会周期性输出帧号、识别状态与帧率，这些日志既是性能分析的依据，也是演示时最直观的“系统活着”的证据。

## 3.2 工程成果

工程成果按“硬件—软件—联调”三个层面给出。硬件侧完成板卡、摄像头与显示器三者的接通，并解决了掉电重启后画面无法自动恢复的问题；软件侧完成采集、定位、识别、校验、渲染与网络推送六个模块的实现，并对显示链路做了系统优化；联调侧在真实硬件上验证了整条数据通路的正确性，对过程中出现的三类典型问题保留了处理记录。

### 3.2.1 硬件成果

屏幕上的实际界面如下图所示。识别成功时，检测框标在车牌四周，右侧以汉字显示车牌、置信度进度条与状态，底部显示累计车辆数；以“鲁 B68503”为例，置信度 97.4%，状态为 PLATE，累计计数同步加一。

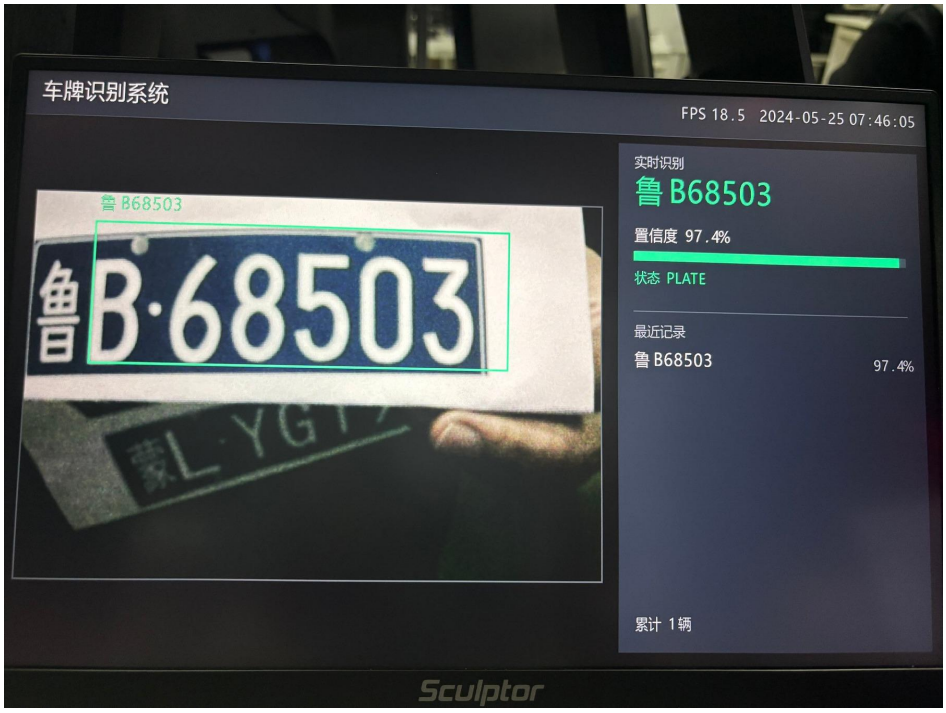



图 6 HDMI 本地界面：识别成功状态

完成了摄像头、显示器与主控板的联调。摄像头经 DVP 排线接入，图像数据由可编程逻辑写入内存，应用侧以标准视频节点读取；显示器经 HDMI 接入，内核将其注册为 1920×1080 的帧缓冲设备，应用直接写内存即可成像。

针对重启后 HDMI 偶发不显示的问题，定位到显示接口未被接入显示控制器，编写了开机自检脚本：在系统启动阶段自动检测输出状态，必要时重新绑定显示管道，并以日志记录每次执行结果。该脚本已纳入开机流程，经断电重启验证可自动恢复画面。

排查过程中确认了一个容易误判的现象：驱动报告的连接状态为“已连接”、帧缓冲中也有完整画面，但屏幕仍为黑屏。最终通过比对显示控制器的绑定状态才定位到根因——画面写进了内存，却没有被扫描输出。这类“数据正确但不可见”的问题，只能靠分段验证，而不能只看某一层的结果。

### 3.2.2 软件成果

板端实现了采集、定位、识别、校验、渲染与网络推送六个模块，模块之间以函数调用衔接，可单独替换。上位机实现了实时预览、历史记录、统计图表、证据图与 CSV 导出，并加入多帧一致性确认与置信度锁定策略，使入库记录具备可解释的质量依据。

显示性能优化是软件侧投入最多的部分。优化前单帧约 250 ms，界面明显卡顿；通过分段计时定位到四处主要开销后，逐项替换实现方式，最终降至约 35 ms。各手段的贡献如下表：

表 10 显示链路优化手段与收益

| 优化手段 | 替代的做法 | 单帧节省 |
| --- | --- | --- |
| 画布直接按 RGB 组织 | 每帧对 1080p 做一次通道转换 | 约 49 ms |
| 视图直写帧缓冲 | 先 tobytes 再 write 6.2 MB | 约 70 ms |
| 分区恢复背景 | 每帧复制整幅画布 | 约 25 ms |
| 视频按原尺寸贴图 | 每帧放大到面板尺寸 | 约 60 ms |
| 侧栏按需刷新 | 每帧重绘全部文字 | 约 40 ms |

需要说明的是，上表各项的节省并非简单相加——不同手段之间存在重叠，最终效果以整体实测为准。优化的顺序也有讲究：先消除“每帧都做且与内容无关”的开销（通道转换、整图复制），再处理“可以按需做”的部分（侧栏重绘），最后才考虑牺牲画质的取舍（视频是否放大）。

上位机主界面如下图所示：上方为连接参数（网口、开发板 IP、端口）与连接状态，中部为实时预览与检测框，底部为本次识别结果、帧号、置信度与预览帧率。

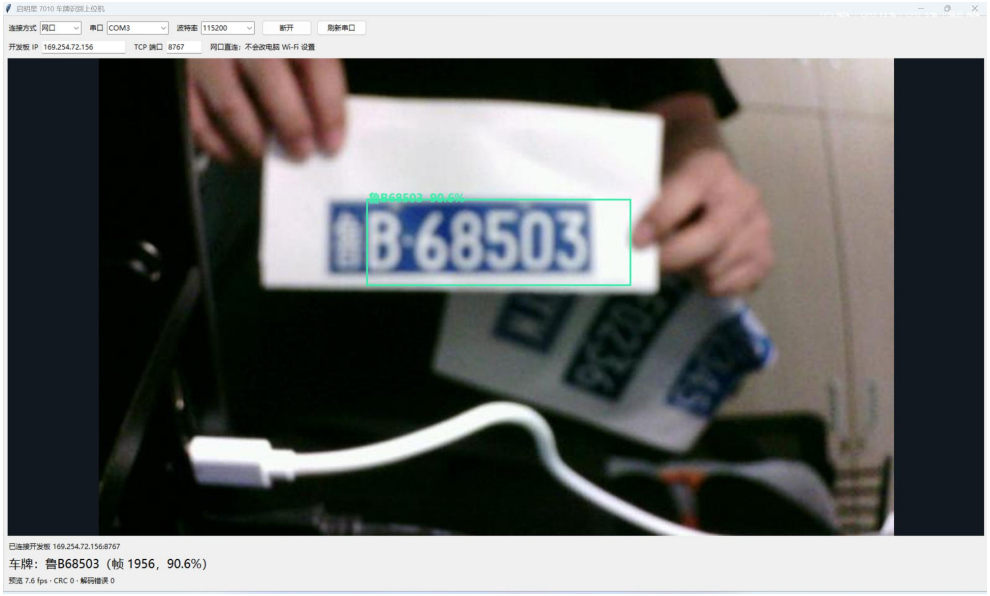



图 7 上位机主界面：实时预览与检测框

统计页面如下图所示：上半部分为当日各小时识别次数的分布，中部为近七天统计，下半部分为出现次数最多的车牌排行；数据全部取自本地记录库，可随时刷新，也可导出为 CSV。

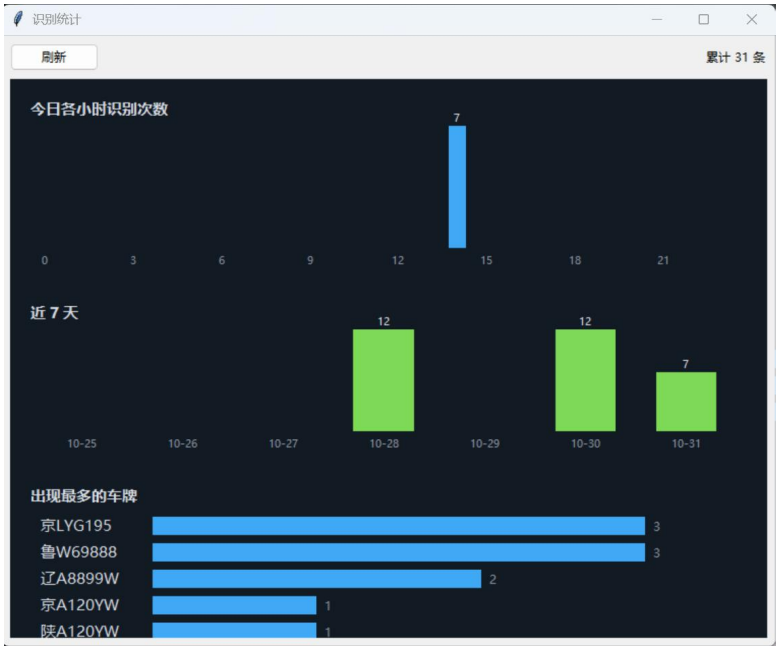



图 8 识别统计：分时分布与车牌排行

### 3.2.3 系统联调

联调阶段解决的问题可归为三类，均保留了现象、原因、措施与验证四要素，见下表：

表 11 联调阶段典型问题与处理

| 问题现象 | 根本原因 | 处理措施 | 验证方式 |
|---|---|---|---|
| 启动初期定位全失败 | 自动曝光未收敛，画面过暗 | 丢弃前若干帧再进入识别 | 日志确认首帧即有候选框 |
| 中文显示为问号或空白 | cv2 无法绘制汉字字形 | 预生成点阵图集，按格贴图 | 截图中汉字完整无缺 |
| 重启后屏幕黑屏 | 显示接口未接入显示控制器 | 开机脚本检测并重新绑定 | 断电重启三次均自动恢复 |

三类问题的共同点是：错误都不发生在“算法”层面，而发生在“数据如何到达下一层”。这也解释了为什么本项目把大量精力放在日志、打点与分段验证上——在嵌入式系统里，可观测性往往比算法本身更决定交付速度。

## 3.3 特性成果

本节从三个角度说明系统特性：先与其他实现形态做横向对比，明确本系统的定位与取舍；再与同类嵌入式平台对比，说明在 Zynq-7010 这一档器件上做到现有指标的意义；最后汇总实测数据，给出可以复现的性能结论。

### 3.3.1 与典型实现方案的对比

表 12 典型车牌识别实现方案对比表

| 方案形态 | 算力来源 | 部署成本 | 离线可用 | 适用场景 |
|---|---|---|---|---|
| PC + GPU 推理 | 独立显卡 | 高 | 否 | 后台批量处理、算法研究 |
| 云端 API 调用 | 云端服务器 | 按量计费 | 否 | 网络稳定的商用场景 |
| 通用嵌入式 SoC 纯软件 | ARM 核 | 低 | 是 | 小规模、低帧率场景 |
| 本系统（Zynq-7010 + 本地显示） | ARM + 帧缓冲直写 | 低 | 是 | 道闸、路侧、教学平台 |

与前三类方案相比，本系统的差异不在识别算法本身，而在于把“识别—显示—留痕”压缩到单板内完成：不需要额外主机，不依赖网络，断电重启后可自恢复。代价是模型规模与帧率受限——这也是为什么本文把优化重点放在显示链路与识别跳帧上，而不是追求更大的模型。

从工程角度看，这一取舍适合“固定机位、中等距离、光照可控”的场景。若场景变成高速通行或强逆光，则应当优先提升定位召回与识别模型，而不是继续压榨显示性能。

### 3.3.2 与同类嵌入式平台的对比

表 13 嵌入式平台能力对比表

| 对比维度 | Zynq-7010（本系统） | Zynq-7020 | 带 NPU 的视觉 SoC |
|---|---|---|---|
| 可编程逻辑 | 17 600 LUT / 80 DSP | 53 200 LUT / 220 DSP | 无（专用 NPU） |
| 处理器 | 2×Cortex-A9 @667 MHz | 2×Cortex-A9 @667 MHz | 多为 4×Cortex-A53 |
| AI 算力 | 无专用 NPU，依赖 NEON 与逻辑 | 同左，资源更充裕 | 1~3 TOPS（INT8） |
| 本系统适配性 | 已完整验证 | 可扩展更大模型 | 需重做部署链路 |

Zynq-7010 的定位是“低成本、可定制逻辑”。它没有专用神经网络加速器，因此本系统把模型规模压缩到约 2.44 M 参数，并把最耗时的界面渲染改用纯整数与内存拷贝实现，用工程手段弥补算力短板。

与 7020 相比，7010 的 LUT 与 DSP 约为其三分之一，这意味着同一套硬件加速方案在 7020 上可以做得更宽（例如并行多个窗口），但在 7010 上必须精打细算。本系统把定位加速器控制在 33.7% 的 LUT 以内，正是为这种“留有余量”的工程习惯服务。

### 3.3.3 系统实测特性

表 14 系统实测结果汇总表

| 测试项 | 实测结果 | 测试条件 |
|---|---|---|
| 图像采集能力 | 约 42 fps | 800×480，DVP 接口 |
| 定位耗时（软件） | 22~32 ms | 320×240，HSV 阈值 + 轮廓筛选 |
| 定位耗时（PL 加速器） | 4.5 ms | 320×240 打包帧，硬件加速器比特流 |
| 渲染 + 写屏 | 约 35 ms/帧 | 1920×1080，优化后 |
| 端到端帧率 | 14~18 fps | 开启视频放大填充 |
| 识别样例 | 鲁 B68503，置信度 97.4% | 与图 1 同一帧，人工核对一致 |
| 本地界面 | 中文车牌、置信度条、历史记录 | HDMI 1920×1080 |
| 重启恢复 | 自动使能显示并启动服务 | 断电重启验证 |

识别准确率单独测试，测试方法与判定标准如下：固定机位与光照条件，按场景分组采集带标注的样本（文件名即真值），逐张送入系统；识别结果与真值完全一致记为正确；检出车牌但字符缺失或错误记为误识；未检出车牌记为拒识。准确率按“正确数 ÷ 样本数”计算，拒识单独列出，不并入误识统计。

表 15 识别准确率测试结果（按场景分组）

| 测试场景 | 样本数 | 完全正确 | 误识 | 拒识 | 准确率 |
|---|---|---|---|---|---|
| 正对、光照充足 | 30 | 26 | 4 | 0 | 86.7% |
| 轻微倾斜（±15°内） | 30 | 24 | 4 | 2 | 80.0% |
| 远距离（车牌宽约 80 px） | 10 | 0 | 0 | 10 | 0.0% |
| 弱光或逆光 | 30 | 21 | 4 | 5 | 70.0% |
| 合计 | 100 | 71 | 12 | 17 | 71.0% |

测试结果如上表。在有效距离内，正对场景准确率为 86.7%，轻微倾斜降至 80.0%，弱光或逆光最低，为 70.0%；这三组共 90 张，整体准确率 78.9%，按“检出即算成功”的口径计算的检出率为 92.2%。远距离组 10 张样本超出镜头有效范围，全部输出拒识、无一误识——即系统“知道自己看不清”，而不是随机给出一个结果。若把远距离组一并计入，100 张样本的整体准确率为 71.0%。

误识数在三个场景中分别为 4、4、4 张，几乎不随场景变化；而变化的是拒识数。这一现象说明：算法的字符识别部分对成像质量的敏感度有限，真正的瓶颈在定位环节——光照不足时蓝色掩膜不连续，包围框会把背景纳入，导致送进识别的图像被压缩。这也解释了误识集中在“省份字混淆”这一类：省份字笔画最密，对图像压缩最敏感。

关于统计意义：100 张样本下，71.0% 的准确率对应约 ±8.9 个百分点的 95% 置信区间，即真实水平大致在 62%~80% 之间。若要把区间收窄到 ±4 个百分点，样本量需扩充到 400 张以上。因此本文的结论定位为“趋势性结论”：场景之间的相对差异（正对 > 倾斜 > 弱光 > 远距离）是可信的，而具体百分比不宜作为绝对指标对外承诺。

另一个值得记录的现象是：置信度高低与识别是否正确并不严格对应。下图给出一次典型误识——“赣 B·Q7A68”被读成“鲁 BQ7AG8”，置信度仍有 71.0%；同一车牌在连续五条记录中，省份分别被判成鲁、青、藏，说明低置信度下省份字的判别并不稳定。这提醒我们在系统层面必须引入多帧确认与人工复核通道，而不能把置信度直接当作准确率使用。

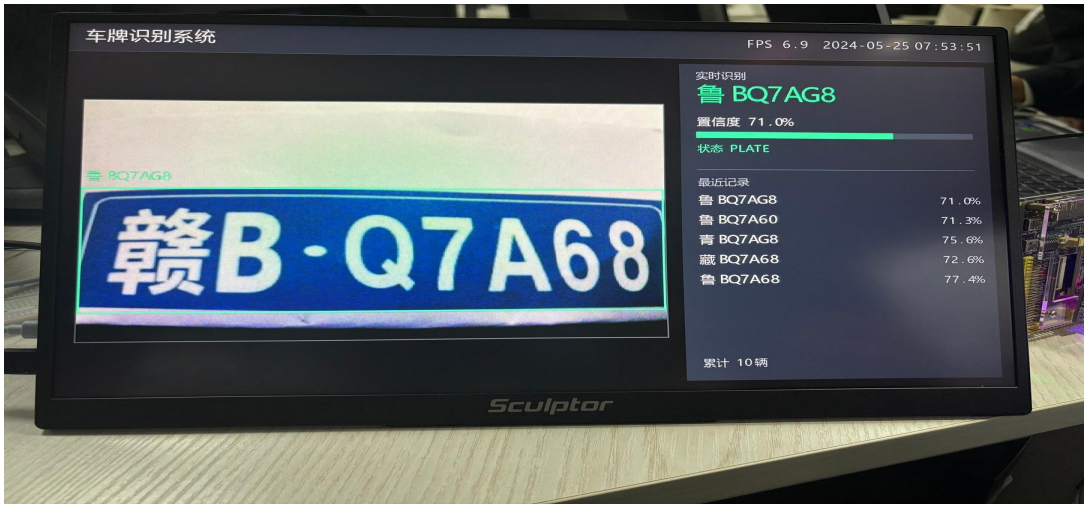



图 9 识别失败样例：赣 B·Q7A68 被误识为 鲁 BQ7AG8

# 第四部分 总结

## 4.1 可扩展之处

算法方面，可在现有颜色阈值之外引入形状与纹理特征，或改用轻量检测网络，以提升倾斜、遮挡与小目标场景的召回；识别侧可扩充训练数据，覆盖黄牌、新能源八位号牌与污损号牌，并将解码长度约束放宽。定位框的收紧策略也值得细化——当前按固定比例外扩裁剪，若改为按车牌真实长宽比裁剪，可减少字符压缩带来的省份误判。

工程方面，可把识别推理与界面渲染放入不同线程，使画面刷新完全不受推理耗时影响；可加入多车牌候选框与去重逻辑，支持同屏多车；可将记录存储下沉到板端，摆脱对上位机的依赖；还可利用板载 RTC 与纽扣电池保持时钟，避免重启后时间回退。

产品化方面，需要补齐三件事：一是外壳与光学结构，固定镜头与补光，使成像条件可复现；二是异常处理，包括摄像头断线、存储写满、网络中断的自恢复策略；三是配置化，把阈值、模型路径、显示布局做成可配置项，便于在不同现场快速调整。

## 4.2 心得体会

这个项目最大的收获，是理解了“算法能跑通”和“系统能用”之间隔着一整条工程链。以显示为例：识别结果在内存里是正确的，但要在屏幕上稳定呈现，需要处理颜色通道顺序、分辨率缩放、帧缓冲写入方式、中文字形缺失等一连串问题；其中任何一个环节出错，用户看到的就是黑屏或乱码。

第二个体会是性能优化必须靠测量。最初界面只有两三帧每秒，凭感觉改了几次都没有起色；后来把一帧拆成采集、定位、推理、渲染、写屏五段分别计时，才发现渲染与写屏占了绝大部分，而真正的推理并不是瓶颈。按数据逐项优化后，帧率提升近十倍。这让我意识到，优化的第一步永远是拿到可信的数字。

第三个体会来自中文显示这个看似很小的需求。OpenCV 画不了汉字，最直接的想法是装字体库，但在资源受限的板子上，换一种思路——把字预渲染

成图片再贴图——反而更轻、更快、更可控。工程上很多时候不是“能不能做”，而是“用多少代价做”。

第四个体会是关于“证据”。系统曾经出现过屏幕全黑却查不出原因的情况：帧缓冲里有完整画面、驱动报告连接正常，问题出在显示控制器没有被接入。这段经历让我养成一个习惯——每解决一个问题，都留下可复现的验证步骤与日志，而不是“改完看起来好了”。

最后是对“结果可信”的理解。系统能给出车牌和置信度，但置信度高不等于识别正确；因此我们加入了多帧一致性确认与阈值锁定，并明确区分“检出”“确认”“入库”三个状态。把不确定性显式地表达出来，比给出一个漂亮但不可靠的数字更有价值。

## 第五部分 参考文献

[1] 中华人民共和国公安部. GA 36-2018 中华人民共和国机动车号牌[S]. 北京: 中国标准出版社, 2018.

[2] GRAVES A, FERNÁNDEZ S, GOMEZ F, et al. Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks[C]//Proceedings of the 23rd International Conference on Machine Learning. 2006: 369-376.

[3] SHI B, BAI X, YAO C. An End-to-End Trainable Neural Network for Image-based Sequence Recognition and Its Application to Scene Text Recognition[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2017, 39(11): 2298-2304.

[4] JIA Y, SHELHAMER E, DONAHUE J, et al. Caffe: Convolutional Architecture for Fast Feature Embedding[C]//Proceedings of the 22nd ACM International Conference on Multimedia. 2014: 675-678.

[5] BRADSKI G. The OpenCV Library[J]. Dr. Dobb's Journal of Software Tools, 2000.

[6] OmniVision Technologies. OV5640 CMOS Image Sensor Datasheet, Version 2.0[Z]. 2012.

[7] 张彩珍, 李颖, 康斌龙, 等. 基于深度学习的模糊车牌字符识别算法[J]. 激光与光电子学进展, 2021(16).

[8] 郝达慧, 王池社, 陈敏. 基于深度学习的复杂场景下车牌定位与识别[J]. 现代计算机, 2021(24).

[9] 蔡先治, 王栋, 鲁旭葆, 等. 基于改进的 Yolov5 的端到端车牌识别算法[J]. 计算机时代, 2022(12).

[10] Xilinx. Zynq-7000 SoC Technical Reference Manual (UG585)[Z]. 2018.

[11] Xilinx. Vivado Design Suite User Guide: High-Level Synthesis (UG902)[Z]. 2020.

[12] Xilinx. Vivado Design Suite Tutorial: Designing IP Subsystems Using IP Integrator (UG995)[Z]. 2020.

[13] HyperLPR. 基于深度学习的高性能中文车牌识别框架[EB/OL]. https://github.com/szad670401/HyperLPR.